

Disclaimer: Dieses OER-Dokument stammt aus dem Projekt [proTirone](#), das über GitHub [freie Unterrichtsmaterialien](#) samt [Quellcode](#) offeriert. proTirone-Materialien werden unter den Bedingungen der [CC-BY-4.0-Lizenz](#) so angeboten, wie sie sind, ohne Zusage bestimmter Eigenschaften und ohne Gewährleistung (§5). Dafür dürfen sie – bei angemessener Namensnennung (§3) – für beliebige (auch kommerzielle) Zwecke verändert und weitergegeben werden (§2).

LF09:17:ssh, Tunnels und VPNs

1. Secure Shell (ssh) in der Theorie [→ ZP:Sheet2]

- **ssh** ist ein Client-Serversystem mit asymmetrischer Verschlüsselung zur Übertragung von Daten, das auf Layer VII arbeitet.
- Jeder **ssh-Client** und jeder **ssh-Server** hat sein je eigenes rechner spezifisches Schlüsselpaar, bestehend aus öffentlichem und privatem Schlüssel: `ls /etc/ssh/ssh_host_rsa_key.pub`.

Prinzipieller Ablauf der Datenübertragung:

- **secure-shell-Client**
 - 1. agiert als Shell auf dem Clientrechner,
 - 2. stellt ssh-Verbindung zum Server her,
 - 3. nimmt einen Kommandozeilenbefehl (mit Parametern) entgegen,
 - 4. verschlüsselt ihn und
 - 5. überträgt ihn zum *secure-shell-Server*
- **secure-shell-Server**
 - 1. hilft bei der Herstellung der Verbindung
 - 2. empfängt danach einen verschlüsselten Kommandozeilenbefehl,
 - 3. entschlüsselt ihn,
 - 4. lässt in von einer Shell ausführen,
 - 5. übernimmt deren Output,
 - 6. verschlüsselt den Output
 - 7. und sendet ihn als Antwort auf den Request zurück.

Varianten:

- **scp**-Client nimmt einen Dateipfad + einen Zielpfad und überträgt den Dateiinhalte verschlüsselt zum ssh-Server.

- **scp**-Server entschlüsselt den Dateiinhalt und der Zielpfad und lässt den Inhalt unter dem Zielpfad auf der ssh-Server-Rechner abspeichern.
- **sftp-Clientsubsystem**
 - nimmt ftp-Befehle an der lokalen Shell entgegen,
 - interpretiert diese lokal
 - verschlüsselt das Ergebnis + die remote auszuführenden Befehle
 - sendet beides zum ssh-Server.
- **sftp-Serversubsystem**
 - entschlüsselt die erhaltenen Befehle
 - führt sie bei sich lokal = remote aus.

Konkreterer Ablauf der Datenübertragung (in einer leicht abgeschwächten Version):

1. User X ruft ssh-Client mit Adressat-IP (und Adressat-User Y) in einer lokalen Shell auf.
2. Die lokale Shell lässt den ssh-Client ausführen und zeigt dessen Output an.
3. ssh-Client sendet Verbindungswunsch zum ssh-Server.
4. ssh-Server schickt Hashwert seines öffentlichen Schlüssels zurück.
5. Kennt der ssh-Client den Hash nicht (= noch nicht in der Datei `knownhosts` enthalten) fragt der ssh-Client bei User X nach, ob er der Verbindung traut. Wenn nicht, beendet er die Kommunikation.
6. Akzeptiert User X die Verbindung oder kennt der ssh-Client den Server, sendet der ssh-Client den System-eigenen öffentlichen Schlüssel (`/etc/ssh/ssh_host_rsa_key.pub`) zum ssh-Server.
7. Erhält der ssh-Server einen öffentlichen Schlüssel eines Clients, vermerkt er den für die Session und sendet seinen System-eigenen öffentlichen Schlüssel zurück.
8. Der ssh-Client fordert den User zur Eingabe 'seines' User-Namens und User-Passworts auf dem ssh-Server-System auf.
9. Der ssh-Client verschlüsselt Usernamen und Passwort mit dem öffentlichen Schlüssel des ssh-Server-Systems.
10. Der ssh-Client sendet den verschlüsselten Usernamen+Passwort an den ssh-Server
11. Der ssh-Server entschlüsselt den verschlüsselten Usernamen+Passwort
12. Der ssh-Server übergibt die Klardaten an das lokale Authentifizierungs- / Loginsystem.
13. Akzeptiert die lokale Authentifizierungs- / Loginsystem die Eingabe, akzeptiert ssh-Server die Verbindung zum ssh-Client (sonst nicht)

14. LOOP:

- ssh-Client:
 - nimmt ein Kommandozeilenbefehl des Users-X (mit Parametern) entgegen
 - **verschlüsselt** das Kommando **mit** dem **öffentlichen Schlüssel des ssh-Server-Systems**
 - sendet das verschlüsselte Kommando zum ssh-Server
- ssh-Server:
 - empfängt den verschlüsselten Kommandozeilenbefehl
 - **entschlüsselt** ihn **mit** dem **privaten Schlüssel des ssh-Server-Systems**
 - lässt das Kommando von einer lokalen Shell auf dem ssh-Server mit den Rechten vom User Y ausführen
 - nimmt den Output des Kommandos (in der lokalen Shell) entgegen
 - **verschlüsselt** den Output **mit** dem **öffentlichen Schlüssel des ssh-Client-Systems**
 - sendet den verschlüsselten Output zum ssh-Client
- ssh-Client:
 - empfängt den verschlüsselten Kommandooutput
 - **entschlüsselt** ihn **mit** dem **privaten Schlüssel des ssh-Client-Systems**
 - übergibt den entschlüsselten Output zur Darstellung an die lokale Client-Shell

Anmerkungen:

- In noch feinerer Granularität enthält die Schlüsselaustauschphase - wie TLS/SSL - eine Absicherung vor Man-in-the-Middle-Angriffen.

2. Exkurs in Sachen Rechtemanagement

s. [lf.cx/cx.access-mangement](#)

3. Secure Shell (ssh) in der Praxis

ÜBUNG LF09/17:SSH:01

Sie erhalten einen

- einen Ubuntu 25.04 Serverrechner mit
 - einem User `gsldkm` mit Passwort `.gsldkPw4all` und

- einem User `root` mit Passwort `lgsldkPw4all`
- und für das Schul-Wlan einen WLAN-User `fius` mit Passwort `fachinformatikeruser`.

Organisieren Sie Server und Client so, dass Sie sich per ssh vom Client auf den Server einloggen können.

1. Ermitteln Sie bitte für alle Teilschritte zuerst, “wie man das macht”,
 2. setzen Sie dann den gewonnenen Plan um und
 3. dokumentieren und erläutern Sie Ihre Schritte in einer Markdown-Datei:
- ☐ Checken und aktivieren Sie ggfls. einen ssh-Server auf dem Serverrechner.
 - ☐ Binden Sie den ssh-Server ins Schul-Wlan ein.
 - ☐ Checken und aktivieren Sie ggfls. einen ssh-Client auf Ihrem Arbeitsrechner (W11, MACOS oder WSL).
 - ☐ Erzeugen Sie mit den richtigen Rechten einen `.ssh`-Ordner in Ihrem Homeverzeichnis.
 - ☐ Erzeugen Sie ein (neues) eigenes ssh-Schlüsselpaar.
 - ☐ Organisieren Sie für den User auf Ihrem Client-Rechner einen Ssh-Zugriff auf den ssh-Server
 - ☐ Loggen Sie sich von ihrem Client-Rechner per ssh auf den ssh-Server ein.
 - ☐ Erkunden Sie mit `find`, `grep` und `ls -la` die Umgebung auf dem ssh-Server.
 - ☐ Legen Sie per remote-Befehl im Homeverzeichnis des ssh-users auf dem ssh-Server einen Ordner mit dem Namen Ihrer Klasse an. (`mkdir -p wieauchimmer`)
 - ☐ Erzeugen Sie per remote-Befehl eine Datei, die Ihrem Vornamen entspricht (nur 7bit-Ascii)
 - ☐ Bitte ermitteln Sie, wie die Verschlüsselung der Kommunikation von Client und Server sichergestellt wird, wenn Sie für Ihren (Client- &| Server-)User noch gar kein Schlüsselpaar generiert haben.
 - ☐ Bitte ermitteln Sie die Schwäche des Verfahrens bis hier hin!

Lösung:

Step 1 / LNX-Server: ssh-Zugang vorbereiten.

- ☐ Paket `openssh-server` installiert? Check per
 - `apt list --installed | grep openssh` oder
 - `dpkg -l | grep openssh`
- ☐ Ist der Server gestartet? Check per
 - `netstat -ano | grep tcp | grep 22` oder
 - `sudo lsof -i -n | grep ssh` oder
 - `sudo systemctl status ssh`
 - Ggfs. starten mit `sudo systemctl start ssh`

- ☐ `.ssh` im Homeverzeichnis? Ggfs. anlegen! (`mkdir .ssh`)
- ☐ `.ssh` mit Rechten 700? Ggfs. umwidmen per `chmod 700 .ssh`
- ☐ `.ssh` leer? (ggfs. alle Dateien löschen per `rm .ssh/*`)
- ☐ Leere Datei `authorized_keys` anlegen mit `touch .ssh/authorized_keys`
- ☐ Schreib- und Leserechte auf `user-only` setzen per `chmod 600 .ssh/authorized_keys`

Step 2.A / LNX-Client: *ssh*-Nutzung vorbereiten.

- ☐ Paket *openssh-client* installiert? Check per
 - `apt list --installed | grep openssh` oder
 - `dpkg -l | grep openssh`
- ☐ `.ssh` im Homeverzeichnis? Ggfs. anlegen! (`mkdir .ssh`)
- ☐ `.ssh` mit Rechten 700? Ggfs. umwidmen per `chmod 700 .ssh`

Step 2.B / W11-Client: *ssh*-Nutzung vorbereiten.

- ☐ Rechte lassen wie sie sind. (Zur Nutzung der user-keys wird später `ssh-add` ausgeführt)
- ☐ Ist der (open)ssh-Client installiert? Wenn `ps ssh ->` Ausgabe, dann ja. Wenn nein, bitte installieren.
- ☐ Ist ein Ordner `.ssh` im eigenen Homeverzeichnis angelegt? Wenn nicht (`$ ls .ssh !=` ' ') dann lege den Ordner ggfls. an: `$ mkdir .ssh`

Step 3 LNX/W11-Client:

- ☐ *ssh*-Client in einer Shell aufrufen.
- ☐ Bei Nachfrage des Clients die auf dem Server zu nutzende UserID eingeben
- ☐ Bei Nachfrage des Clients das Passwort des auf dem Server zu nutzenden Users eingeben
- ☐ Erkunden und Dateien anlegen.

Step 4 Die genutzten Schlüssel:

- Der SSH-Server hat ein Rechner-spezifisches, systemweit gültiges Schlüsselpaar
- LNX: `ls /etc/ssh/ssh_host_rsa_key.*`
- W11: `ls C:\ProgramData\ssh\*`

Verabredung eines gemeinsamen Schlüssels nach Diffie-Hellman-Schlüsselaustausch

1. <https://de.wikipedia.org/wiki/Diffie-Hellman-Schl%C3%BCsselaustausch>
2. Server nutzt dabei sein Rechner-spezifisches, systemweit gültiges Schlüsselpaar
3. Client erzeugt ein einmal-Ssh-Schlüssel-Paar, sofern der User noch kein User-spezifisches Schlüsselpaar besitzt.

Step 5 Nachteil des Verfahrens insoweit:

Alle Schülerinnen müssen Server-User und dessen Passwort kennen und immer wieder nutzen.

ÜBUNG LF09/17:SSH:02

Automatisieren Sie jetzt ihren Zugang so, dass Sie das Passwort des Users auf dem Server nicht mehr eingeben müssen.

- ☐ Erzeugen Sie sich mit `ssh-keygen` ein eigenes Schlüsselpaar.
- ☐ Transferieren Sie Ihren neue generierten öffentlichen Schlüssel an die richtige Stelle auf dem ssh-Server-Rechner.
- ☐ Loggen Sie sich wieder aus und wieder ein. Beschreiben Sie den Unterschied.

Lösung:

• LNX

- ☐ wechsele in das `.ssh`-Verzeichnis, rufe in einer shell `$ ssh-keygen` auf und gib, wenn danach gefragt
 - * den gewünschten Namen der Schlüsseldatei ein,
 - * dann eine (ggfls. leere) Passphrase für den Schlüssel
 - * und wiederhole die (ggfls. leere) Passphrase noch einmal
- ☐ Lass Dir das in `.ssh` angelegte Schlüsselpaar auflisten `$ ls .ssh*`
 - ☐ Variante 1: Lass mit `ssh-copy-id -i .ssh/${YOURKEY}.pub user@ssh-server-ip` Deinen public-Key zum Server übertragen und in die Datei `authorized_keys` Deines Server-Users eintragen.
 - ☐ Variante 2:
 - ☐ lassen per `scp` den Public-Key zum Server übertragen.
 - ☐ Logge Dich auf den SSH-Server ein.
 - ☐ Hänge ihn mit `cat youpubkey >> .ssh/authorized_keys` an die bestehenden an.

• WIN

- ☐ Öffne Powershell (pwsh), wechsele in das Homeverzeichnis: `ps> cd $HOME\.ssh`, lass mit `ssh-keygen` ein Schlüsselpaar erzeugen und gib, wenn danach gefragt,
 - * den gewünschten Namen der Schlüsseldatei ein,
 - * dann eine (ggfls. leere) Passphrase für den Schlüssel

- * und wiederhole die (ggfls. leere) Passphrase noch einmal
- ☐ Lass Dir das in .ssh angelegte Schlüsselpaar auflisten `ps> ls .ssh*`
- ☐ Kopiere die Pub-Datei in Homeverzeichnis des SSH-Users auf dem SS-Server (`ps> scp ${YOURKEY}.pub userY@${YourServerIP}:`)
- ☐ Logge Dich auf den SSH-Server ein.
- ☐ Hänge ihn mit `cat youpubkey >> .ssh/authorized_keys` an die bestehenden an.
- ☐ Mache Deinen Schlüssel auf dem Clientrechner bekannt
 - ☐ `Get-Service ssh-agent -> stopped`
 - ☐ `Start-Service ssh-agent -> started`
 - ☐ `Get-Service ssh-agent -> running`
 - ☐ `ssh-add $env:USERPROFILE\.ssh\${YourSshKey}`

- ALL: Logge Dich auf der ssh-Server aus und wieder ein: Du wirst ohne Passwortfrage eingeloggt.

Hinweis: Wenn Sie die Passphrase mit einem Passwort belegen, verschlüsselt der Client Ihren privaten(!) Schlüssel damit und fragt Sie lokal vor dessen Nutzung jeweils nach dieser Passphrase. - Vorteil: Wenn der private Schlüssel Ihres Clients gestohlen wird, braucht der Dieb, um als Sie auf dem Server zu agieren, immer noch die Passphrase. - Nachteil: Sie müssen nun die Passphrase anstatt des Userpassworts angeben. Zur Automatisierung braucht man aber oft eine 'unterbrechungsfreie' Kommunikation. Diesen 'Nachteil' können Sie dann mit dem `ssh-agent` auflösen.

4. Tunneling per ssh

ÜBUNG LF09/17:SSH:03

Bringen Sie folgende Beschreibung eines Tunnels von Heise mit Ihrem Rechner als Client und dem Tuxedo-Server zum Laufen.

- `ssh gslDKM@SERVERIP -L 9000:SERVERIP:80`

“Zunächst wird eine ganz normale SSH-Verbindung mit dem Nutzer”gslDKM“ zum SSH-Server aufgebaut. Die Option “L” leitet dann den Tunnel ein: Zunächst folgt der lokale Port “9000”, dann die Adresse des Webservers “SERVER:80”. Es folgt die Passwortabfrage. Anschließend können Sie im Browser auf dem lokalen Rechner “127.0.0.1:9000” eingeben und bekommen zu sehen, was auch unter “SERVER:80” zu sehen ist. (vgl. <https://www.heise.de/tipps-tricks/SSH-Tunnel-nutzen-so>

[geht-s-4320041.html](#))

ÜBUNG LF09/17:SSH:04

Steigern Sie jetzt die Komplexität

1. Erzeugen Sie von Ihrem Rechner zum Tuxedo-Server einen Tunnel.
2. Holen Sie von dort aber nicht die Tuxedo-eigene-Html-Seite, sondern die vom Lehrerrechner.

Achtung: Recht tricky. Szenario: Sie sitzen hinter einer Firewall, die nur ssh durchlässt, tunneln Ihre Arbeitsaufträge auf einen Ihnen zugänglichen Server vor der Firewall, lassen sich den Inhalt von dort holen und ausliefern.

Viel Spaß beim Grübeln, Recherchieren und Lösen.

5. Komplexes Tunneling

A Zusammenfassung [→ ZP:Sheet3]

Eine SSH-Verbindung folgt dem Client-Server-Model. Aber mehr als singuläres Request:Response:

- 1. Request-Anfrage
- 2. allgemeiner Schlüsselaustausch
- 3. Identitätsabsicherung (gegen Man-In-The-Middle)
- 4. Useridentifizierung (Passwort oder `authorized_keys`)
- 5. Loop:
 - a. Client: Command-Eingabe auf Client-Shell
 - b. Client: Command-Verschlüsselung mit Server-Public-Key
 - c. Client/Server: Übertragung
 - d. Server: Command-Entschlüsselung mit Server-Private-Key
 - e. Server: Command-Übergabe an Server/User-Shell
 - f. Server/User-Shell: Command-Ausführung in Server/User-Shell
 - g. Server/User-Shell: Output/Ergebnis-Übergabe an ssh-Server

- h. Server: Output/Ergebnis-Verschlüsselung mit Client-Public-Key
 - i. Server/Client: Übertragung
 - j. Client: Output/Ergebnis-Entschlüsselung mit Client-Private-Key
 - k. Client: Output/Ergebnis-Anzeige in Client-Shell
-

ÜBUNG LF09/17:SSH:05

1. Rekapitulieren Sie, was der Befehl (I) `ssh gslldkm@SERVERIP` tut.
2. Beschreiben Sie als Schrittfolge, was der Befehl (II) `ssh gslldkm@SERVERIP -L 9000:SERVERIP:80` tut.

- Lösung 1: (A)
 - Lösung 2: [→ **ZP:Sheet4**]
-

ÜBUNG LF09/17:SSH:06

1. Wiederholen Sie die Aufgabe 04 so, dass Sie von Ihrem Rechner aus einen Tunnel zum ssh-Serverrechner aufbauen und sich über ihn die Schulhomepage <https://www.gs-ldk.de/> anzeigen lassen.
-

ÜBUNG LF09/17:SSH:06

0. Bilden Sie 3 Gruppen
 1. Konzipieren Sie je Gruppe eine Software, die die Lösung von 5.2 generalisiert.
 2. Spezifizieren Sie, auf welchen Layern Ihre Lösung arbeiten müsste.
 3. Finden Sie einen Namen für Ihre Lösung
 4. Präsentieren und erläutern Sie Ihre Gruppenlösung Ihren Mitschülerinnen.
-

Fazit Ein Tunnel schaltet sich zwischen Layer VII und Layer IV auf den Server, verschlüsselt die Pakete, sendet sie über den Tunnel zum Client, der entschüsselt sie und macht sie auf dem lokalen Port abrufbar.

A Zur Systematik von VPNs

Virtual Private Network (deutsch „virtuelles privates Netzwerk“; kurz: VPN) bezeichnet eine Netzwerkverbindung, die von Unbeteiligten nicht einsehbar ist:

“Ein VPN oder Virtuelles Privates Netzwerk schafft eine private Netzwerkverbindung zwischen Geräten über das Internet. VPNs werden zur sicheren und anonymen Übertragung von Daten über öffentliche Netze verwendet. Sie funktionieren, indem sie die IP-Adressen der Benutzer maskieren und die Daten verschlüsseln, so dass sie von Leuten, die nicht befugt sind, sie zu empfangen, nicht gelesen werden können.

Alle Netzwerkaufrufe von Client-Rechnern/Applikationen werden

- von VPN-Clients auf dem Client-Rechner abgefangen und zum VPN-Server gesendet, der sie nach Freigaben ins Internet oder Intranet umlenkt.
- getunnelt
- verschlüsselt

FAZIT

Ein VPN-Server nimmt die verschlüsselten Netzaufrufe von seinen VPN-Clients entgegen, entschlüsselt sie und reicht sie an die zuständigen Programme weiter:

- Aufrufe ins Intranet sendet er ins Intranet.
- Aufrufe nach außen sendet er nach außen (sofern erlaubt)
- Die Antworten von innen oder außen nimmt er, verschlüsselt sie wieder und sendet sie an den zuständigen VPN-Client zurück.

Hinweis: das VPN-Server und VPN-Client eigenständige Tools sind, sind sie nicht an PAT (Port Address Translation) und dessen Portanzahl gebunden.

- → <https://www.heise.de/tipps-tricks/SSH-Tunnel-nutzen-so-geht-s-4320041.html>
- → https://de.wikipedia.org/wiki/Virtual_Private_Network
- → <https://aws.amazon.com/de/what-is/vpn/>
- [LNX] 1) vgl. https://www.thomas-krenn.com/de/wiki/OpenSSH_Public_Key_Authentifizierung_unter_Ubuntu
- [WIN] 1) vgl. <https://www.ionos.de/digitalguide/server/konfiguration/windows-11-ssh/>